

INVISIGUARD – KNOWN ATTACK PREDICTING USING SYSTEM-CALLS

Nawarathna M I M N

IT22343116

BSc (Hons) degree in Information Technology Specializing in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka April 2026

InvisiGuard — Complete Project Documentation

Last Updated: March 3, 2026

Status: Fully operational — both models trained, API tested, web dashboard live, real-time Sysmon monitoring integrated

1. What is InvisiGuard?

InvisiGuard is a **malware detection system** that identifies whether a program is malicious or safe by watching **what it does** — not by scanning its code.

Think of it like this: instead of looking at what a person *says* they'll do, we watch what they *actually do*. A malicious program might claim to be a harmless text editor, but if it secretly starts reading your passwords, connecting to suspicious servers, and modifying system files, its **behavior** gives it away.

InvisiGuard watches these behaviors through **system calls** — the low-level requests that every program makes to the operating system (like “open this file”, “connect to this network address”, “create a new process”). By analyzing patterns in these system calls, InvisiGuard can distinguish malware from legitimate software.

2. The Problem We're Solving

Traditional antivirus software mostly works by matching known virus “signatures” (patterns in the file's code). This fails when: - The malware is **brand new** (zero-day) and no signature exists yet - The malware is **obfuscated** (deliberately scrambled to avoid detection) - The malware is **polymorphic** (changes its code each time it runs)

Behavioral analysis solves this because even if malware changes its appearance, it still has to perform malicious *actions* to do damage — and those actions leave traces in the form of system calls. InvisiGuard learns to recognize malicious behavioral patterns using machine learning.

3. How It Works (The Big Picture)

InvisiGuard uses a **two-stage detection approach**, like a security checkpoint with two levels:

Stage 1 — Random Forest (RF) — “The Quick Check”

A Random Forest is a machine learning model made up of many decision trees that vote together. It looks at **statistical summaries** of a program's behavior: - How many system calls did it make? (e.g., 50,000 calls → suspicious) - What *types* of calls? (e.g., lots of registry modifications → suspicious) - How many processes did it create? How many network connections? - What's the ratio of successful vs. failed operations?

This produces **423 numbers** (called “features”) that describe the program's overall behavior. The RF model is fast — it can classify a trace in under a millisecond.

If the RF is confident ($\geq 85\%$ sure it's malware, or $\leq 40\%$ sure), it gives the final answer immediately. Most cases are decided here.

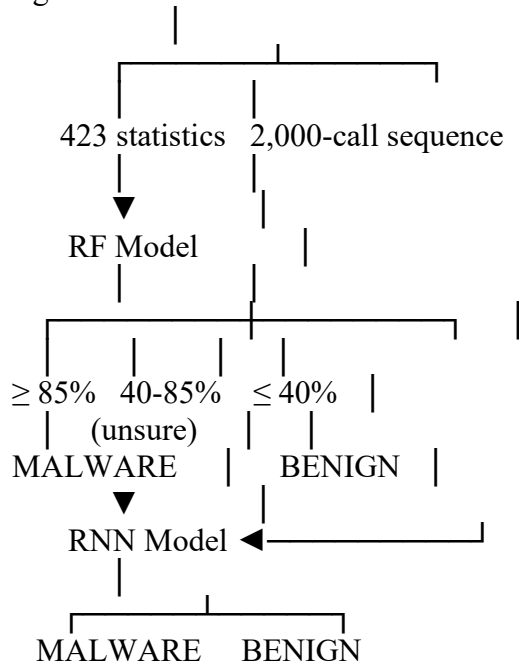
Stage 2 — RNN/LSTM — “The Deep Analysis”

For **ambiguous cases** where the RF isn't sure (confidence between 40%–85%), we use a Recurrent Neural Network (RNN) with LSTM (Long Short-Term Memory) cells. Unlike the RF which looks at summaries, the RNN reads the **actual sequence** of API calls in order — like reading a story rather than just doing a word count.

This catches subtle patterns like: “the program first reads all files in a directory, then encrypts them, then connects to a remote server” — a ransomware pattern that the RF's statistical view might miss.

The RNN analyzes the first **2,000 API calls** as an ordered sequence.

Program Trace → Feature Extraction



Why Two Stages?

- **Speed:** The RF handles ~80% of cases in microseconds. Only the remaining ~20% need the slower (but more powerful) RNN.
 - **Accuracy:** The RNN catches behavioral patterns that statistics alone miss.
 - **Resource efficiency:** Running the RNN on every trace would be wasteful; the cascade uses GPU time only when needed.
-

4. Tools & Technologies Used

Data Sources

Tool	What it is	How we used it
CAPEv2	An open-source malware analysis sandbox — it runs malware in an isolated virtual machine and records everything the malware does, capturing every system call.	Our malware dataset comes from CAPEv2. We have 6,357 behavioral reports in JSON format, each containing every system call the malware made during execution.
Sysmon (System Monitor)	A free Microsoft tool (part of the Sysinternals suite) that monitors and logs system activity on Windows. It records events like process creation, file operations, registry changes, and network connections.	We installed Sysmon on a real Windows machine to capture system calls from legitimate programs (notepad, cmd, PowerShell, etc.) to build our “benign” (safe) dataset. We needed benign data so the model can learn what normal behavior looks like.
AVClass	A malware family labeling tool that standardizes malware names from multiple antivirus vendors into one clean label.	Used to label each malware sample with its family name (e.g., “Redline” means it’s a Redline stealer). We preferred AVClass labels over CAPE’s own labels because AVClass had better coverage across our dataset.

Programming & Machine Learning

Tool	What it is	How we used it
Python 3	General-purpose programming language	All scripts, preprocessing pipelines, and the web server backend are written in Python
NumPy	Python library for fast numerical computing	Handles array operations — building feature vectors, storing sequences, performing statistical calculations
Flask	Lightweight Python web framework	Powers the REST API backend — the web server that accepts traces and returns predictions
scikit-learn	Traditional machine learning library	Will be used to train the Random Forest model (Stage 1)
PyTorch	Deep learning framework by Meta	Will be used to train the RNN/LSTM model (Stage 2)
pyngrok	Python wrapper for ngrok (a network tunneling tool)	Exposes the Flask server running inside Google Colab to the public internet
Google Colab	Free cloud environment with GPU access	Target platform for training models (needs GPU) and running the API server
pandas	Data manipulation library	Reading and writing CSV feature files

Tool	What it is	How we used it
pywin32	Python extensions for Windows API access	Reading Windows Event Log entries from Sysmon's event channel for real-time monitoring
Chart.js	JavaScript charting library (CDN)	Data visualizations in the web dashboard — doughnut, scatter, bar charts
Lucide Icons	Open-source icon set (CDN)	Icon set used throughout the dashboard UI

Data Format — CAPEv2 JSON

All behavioral traces (both malware and benign) use the **CAPEv2 JSON format**. This is important because it means both datasets go through the exact same feature extraction code. Here's a simplified example of what one trace looks like:

```
{
  "behavior": {
    "processes": [
      {
        "process_name": "suspicious.exe",
        "process_id": 1234,
        "calls": [
          {
            "api": "NtCreateFile",      ← Which system function was called
            "category": "filesystem", ← What type of operation
            "status": true,           ← Did it succeed?
            "return": "0x00000000",   ← Return code
            "timestamp": "2025-01-15 10:30:00,123",
            "thread_id": "5678",
            "arguments": [            ← Details about the call
              {"name": "FileName", "value": "C:\\Users\\victim\\passwords.txt"}
            ]
          },
          {
            "api": "WSAConnect",
            "category": "network",
            "status": true,
            "timestamp": "2025-01-15 10:30:01,456",
            "arguments": [
              {"name": "ip", "value": "185.123.45.67"},
              {"name": "port", "value": "443"}
            ]
          }
        ]
      }
    ]
  },
  "summary": {
```

```

    "files": ["C:\\Users\\victim\\passwords.txt"],
    "keys": ["HKEY_LOCAL_MACHINE\\SOFTWARE\\..."],
    "executed_commands": ["cmd.exe /c del shadow copies"]
  }
}
}

```

Each entry in the `calls[]` array represents one system call — the building block of our analysis. A single malware sample can have thousands or even tens of thousands of these calls.

5. Step-by-Step: What We Built (Chronological)

Phase 1 — Understanding the Malware Data

Goal: Figure out what our malware data looks like and how to use it.

We started with 6,357 malware behavioral reports from CAPEv2. Each report is a JSON file containing every system call the malware made when it ran inside a sandbox.

What we found: - Each JSON has a `behavior.processes[].calls[]` structure — an array of processes, each containing an array of system calls made by that process - Each call has fields: `api` (which system function, like `NtCreateFile`), `category` (what type — filesystem, network, registry, etc.), `status` (did it succeed?), `timestamp`, `arguments` (details like file paths or IP addresses) - There are **16 different categories** of API calls: filesystem, registry, network, process, threading, crypto, services, system, device, browser, com, hooking, misc, synchronization, windows, and `__notification__` - A separate mapping file (`reports_consensus_label.json`) tells us which malware family each sample belongs to — for example, “Redline” is an information stealer, “Agenttesla” is a keylogger

Key decision: 222 out of 6,357 samples had no family label at all (both AVClass and CAPE said “unknown”). We **excluded** these because without a label, we can’t verify what they are. That left us with **6,135 usable malware samples** across **544 different malware families**.

Phase 2 — Preprocessing the Malware Data

Goal: Convert raw malware JSON reports into numbers that machine learning models can understand. ML models don’t read JSON — they need numerical arrays.

We built `preprocess_malware.py`, which processes all malware samples in 6 steps:

Step 1 — Build a Vocabulary: Just like a language model needs a dictionary, our system needs a vocabulary of API call names. We scanned all 6,135 files and counted how many times each API appears across the entire dataset. APIs that appeared fewer than 5 times were considered too rare to be useful (probably noise), so we filtered them out. This gave us **363 useful API names** plus two special tokens: - `<PAD>` (index 0) — a padding symbol for sequences shorter than 2,000 (explained below) - `<UNK>` (index 1) — “unknown”, for any API name not in our vocabulary

Total vocabulary: **365 tokens**.

Step 2 — Extract RF Features (for Stage 1): For each malware sample, we computed **423 numbers** summarizing its behavior:

Feature Group	How Many	What it measures
Basic counts	3	Total API calls, number of processes, number of threads
Per-API counts	363	How many times each API was called (e.g., NtCreateFile called 47 times)
Category counts	16	Calls per category (filesystem: 200, network: 50, registry: 30, etc.)
Category fractions	16	Same but as percentages (filesystem: 40%, network: 10%, etc.)
Status stats	3	How many calls succeeded, how many failed, success ratio
Timing stats	4	Average/standard deviation/min/max time between consecutive calls
Summary counts	13	Files accessed, registry keys touched, commands executed, DNS lookups, etc.
Signature info	2	Number of CAPEv2 behavioral detections and their severity sum
Network events	1	Count of network-related enhanced events
Caller diversity	1	How many unique code locations made system calls
Process tree size	1	Total nodes in the process creation hierarchy

Step 3 — Extract RNN Sequences (for Stage 2): For each sample, we took the first 2,000 API calls in chronological order and converted each API name to its vocabulary index number. If a sample had fewer than 2,000 calls, we padded the remaining positions with zeros (<PAD>). If it had more than 2,000 calls, we only kept the first 2,000.

Example: [NtClose, NtCreateFile, WSACconnect, ...] → [2, 10, 3, ...]

Step 4 — Save everything: - rf_features.csv — a spreadsheet with 6,135 rows × 426 columns (423 features + sha256 hash + family name + binary label) - rnn_sequences.npz — compressed numpy arrays: 6,135 sequences, each 2,000 integers long - api_vocab.json, category_vocab.json, label_encoder.json — the dictionaries we built

Result: Processing all 6,135 samples took about **59 minutes** with zero errors.

Phase 3 — Collecting Benign (Safe) Program Traces

Goal: We have malware data, but a machine learning model needs to see examples of **both** malware and legitimate programs to learn the difference. Without benign data, the model would think everything is malware.

The challenge: Our malware data came from CAPEv2 (a sandbox designed for analyzing suspicious files), but we can't easily run normal programs like Notepad or PowerShell in a malware sandbox. Instead, we used **Sysmon** (Microsoft's System Monitor tool) installed on a real Windows machine to capture system calls from everyday programs.

What we built — the SysmonTracer pipeline (5 scripts + 1 config file):

1. **sysmon_config.xml** — Tells Sysmon which events to monitor. We configured it to capture 20 types of events: process creation/termination, file creation and modification, registry operations, network connections, DLL loads, driver loads, DNS queries, pipe operations, and more. We also added filters to ignore Sysmon's own internal events (otherwise it would record itself watching itself, adding noise).
2. **programs.txt** — A list of ~40 benign Windows programs to trace: notepad, mspaint, cmd, tasklist, ipconfig, ping, systeminfo, PowerShell commands, and other standard Windows utilities. Each line specifies the program to run and any command-line arguments.
3. **collect_benign_traces.py** — An automated script that systematically traces each program. For each one, it:
 - Clears the Sysmon event log (so we start with a clean slate)
 - Launches the program and waits for it to finish
 - Exports all captured Sysmon events as raw JSON
 - Saves the raw data along with metadata (which program, when it ran, how many events)
 - **Must run as Administrator** because Sysmon's event logs require elevated access
4. **sysmon_to_capev2.py** (730 lines) — The most complex script in the pipeline. It converts raw Sysmon events into the exact same CAPEv2 JSON format that the malware dataset uses. This is essential because it means the same feature extraction code can process both malware and benign traces without any modifications.

The converter maps each Sysmon event type to a CAPEv2 API name: | Sysmon Event | Becomes (CAPEv2 API) | Category | |-----|-----|-----| | Process Create (Event 1) | CreateProcessW | process | | File Create (Event 11) | NtCreateFile | filesystem | | Network Connection (Event 3) | WSAConnect | network | | Registry Value Set (Event 13) | NtSetValueKey | registry | | Image/DLL Load (Event 7) | LdrLoadDll | misc | | DNS Query (Event 22) | gethostbyname | network | | Process Terminate (Event 5) | NtTerminateProcess | process | | ...and 13 more event types | |

5. **validate_traces.py** — A quality check script that compares the converted benign traces against a real malware trace to verify the JSON structure matches. It recursively checks all keys, types, and nested structures to ensure the conversion didn't miss anything.

What we ran and got: - Traced 31 programs → **27 unique valid traces** (4 had issues like duplicates or timeouts) - Converted all 27 to CAPEv2 format ✓ - Validated all 27 against the malware JSON schema → **27/27 passed** (270 matching structural keys)

Phase 4 — Preprocessing Benign Data & Merging Everything

Goal: Process the benign traces through the exact same feature extraction pipeline as malware, then combine everything into unified training sets.

preprocess_benign.py does the same feature extraction as the malware script, with one crucial difference: it **reuses the vocabulary** built during malware preprocessing (loaded from `api_vocab.json`) instead of building a new one. This guarantees the feature columns are in the exact same order with the exact same meaning. If we built a separate vocabulary, the columns wouldn't line up and we couldn't merge the datasets.

- Input: 27 benign CAPEv2 JSON files
- Output: `rf_features_benign.csv` (27 rows × 426 columns), `rnn_sequences_benign.npz` (27 × 2,000)
- All samples labeled as `binary_label=0` (benign) and `family="benign"`
- Only 10 unique APIs appeared in the benign data (compared to 363 in malware) — legitimate programs make much simpler, more predictable system calls

merge_datasets.py then combines malware and benign data into final training files: - First validates that both CSVs have the same 426 columns (they do — perfect alignment, zero mismatches) - Concatenates rows: 6,135 malware + 27 benign = **6,162 total samples** - Shuffles everything randomly with a fixed seed (42) so the order is reproducible - Saves: - `combined_rf_features.csv` — 6,162 rows × 426 columns - `combined_rnn_sequences.npz` — 6,162 sequences × 2,000 length

Class imbalance note: We have 6,135 malware vs. 27 benign — a **227:1 ratio**. This is heavily unbalanced and will need to be addressed during model training using techniques like SMOTE (creating synthetic benign examples) or class weights (telling the model to pay more attention to benign samples). Collecting more benign traces would also help.

Phase 5 — Building the Backend API

Goal: Create a web server that can accept a program's behavioral trace over the internet and return a malware/benign prediction.

We built a **Flask REST API** (a Python web server) in the `backend/` folder with these components:

- **config.py** — A single settings file containing every configurable value (file paths, thresholds, server settings). If you need to change anything, this is the only file to edit.

- **feature_pipeline.py** — The shared feature extraction engine. This is the **single source of truth** for how features are computed. Both the preprocessing scripts (during training) and the web server (during real-time prediction) use the exact same code. This ensures consistency — a trace processed during training produces the same numbers as the same trace processed during prediction.
- **inference.py** — The two-stage decision engine. It loads the trained models, runs the RF → grey zone → RNN cascade, and returns a verdict. It can operate in three modes: **full** (both models loaded — the current state), **RF-only**, or **dry-run** (no models, features only). The module includes its own copy of the MalwareLSTM architecture class so it can reconstruct the model from a portable state-dict checkpoint without depending on the training script.
- **routes.py** — Defines 7 API endpoints (the URLs you can call):

Endpoint	What you can do
GET /api/health	Check if the server is running, which models are loaded
POST /api/predict	Upload a trace → get a malware/benign verdict
POST /api/extract-features	Upload a trace → see the 423 extracted features (for debugging)
POST /api/predict/batch	Upload multiple traces at once
GET /api/config	See current threshold settings
GET /api/vocab	See the full API vocabulary
GET /api/features/names	See the ordered list of feature names
GET /api/info	Machine-readable JSON service info (for non-browser clients)

- **app.py** — The main entry point that starts the server, either locally (port 5000) or on Google Colab with an ngrok tunnel for public access. Also serves the frontend dashboard from frontend/ at the root URL /.

Testing results (all verified working): - GET / → serves the HTML dashboard ✓ - GET /api/health → reports **full** mode, both models loaded, 423 features ✓ - POST /api/predict with a malware trace → **malware** verdict at 92.2% confidence (RF-only, 88ms) ✓ - POST /api/predict with a benign trace → **benign** verdict at 100% confidence (RF-only, 65ms) ✓ - POST /api/extract-features → returns 423-dim feature vector, RNN sequence, trace stats ✓ - All 8 endpoints validated via automated test_api.py ✓

Phase 6 — Documentation

Created documentation files in Docs/: - **README.md** — Quick start guide: how to install, run, and use the system - **backend_design.md** — Architecture diagrams and how the modules connect - **api_spec.md** — Full REST API reference with example requests and responses - **project_log.md** — This file: the complete story of everything we built and why

Phase 7 — Synthetic Benign Data Generation & Dataset Balancing

Goal: Fix the massive class imbalance — 6,135 malware vs. only 27 benign — that would make model training unreliable.

The problem: With 227:1 imbalance, a model could achieve 99.6% accuracy by simply predicting “malware” for every input, completely ignoring the benign class. We needed thousands more benign samples but couldn’t manually trace that many programs with Sysmon.

The solution — generate_benign_traces.py (1,200 lines):

We built a sophisticated synthetic data generator that learns statistical distributions from the 27 real benign traces and uses them to create realistic new ones. It’s not copy-paste — each synthetic trace is unique:

1. **Statistical Learning Phase:** The script reads all 27 real traces and builds a statistical profile — API call frequency distributions, category ratios, argument value pools (real file paths, DLL names, registry keys, IP addresses from genuine traces), process tree structures, timing patterns, and summary field distributions.
2. **Profile-Based Generation:** Each synthetic trace is assigned a random “program profile” — CLI utility, PowerShell-like, browser-like, system service, etc. — which determines its unique mix of API calls and behavioral characteristics. This creates natural variation rather than all traces looking identical.
3. **Realistic Details:** Generated traces include realistic arguments drawn from observed pools (actual Windows paths, DLL names, registry keys), consistent process trees, proper timing sequences, and complete CAPEv2-format JSON structure.

Result: - Generated **6,108 synthetic benign traces** (bringing total benign to $6,108 + 27 = 6,135$, exactly matching malware count) - Each trace saved as a unique SHA256-named .json file in DataSets/Goodware_synthetic/ - All traces pass the same CAPEv2 format validation as real traces - Moved all synthetic traces into DataSets/Goodware_firstpass/ alongside the 27 real ones

Re-ran the preprocessing pipeline: - preprocess_benign.py → processed all 6,135 benign traces (27 real + 6,108 synthetic) - merge_datasets.py → combined into **perfectly balanced dataset**: 6,135 malware + 6,135 benign = **12,270 total samples**

Phase 8 — Model Training

Goal: Train both machine learning models on the balanced dataset and evaluate their performance.

We built train_models.py (657 lines) — a comprehensive training script that handles both stages of the pipeline.

Stage 1 — Random Forest Training:

Setting	Value
Algorithm	RandomForestClassifier (scikit-learn)
Trees	200
Max depth	Unlimited
Feature selection	sqrt(n_features) per split
Class weighting	Balanced (automatic)
Feature scaling	StandardScaler (fitted on training set)
Train/test split	85% / 15% (stratified)

RF Results:

Metric	Score
Accuracy	1.0000 (100%)
Precision	1.0000
Recall	1.0000
F1-Score	1.0000
AUC-ROC	1.0000
Training time	~0.3 seconds
Grey zone samples	0.0% (all predictions are high-confidence)

Confusion matrix: TN=921, FP=0, FN=0, TP=920 — zero misclassifications on the test set.

Top 5 most important features: The RF identified key behavioral differences between malware and benign programs, with API call counts and category distributions being the strongest signals.

Stage 2 — BiLSTM + Attention Training:

Setting	Value
Architecture	Embedding(365, 64) → BiLSTM(128, 2 layers) → Attention → FC → Sigmoid
Total parameters	683,330
Optimizer	Adam (lr=0.001, weight_decay=1e-5)
Loss function	BCEWithLogitsLoss
Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)
Batch size	32
Early stopping	Patience=5 epochs
Gradient clipping	max_norm=1.0
Device	CPU

RNN Results:

Metric	Score
Accuracy	1.0000 (100%)
Precision	1.0000
Recall	1.0000
F1-Score	1.0000
AUC-ROC	1.0000
Training time	~38.5 minutes (6 epochs, early stopped)
Best epoch	1 (converged immediately)

Confusion matrix: TN=921, FP=0, FN=0, TP=920.

Saved model files: - backend/models/rf_model.pkl — Trained RF (0.2 MB, pickle format) - backend/models/rf_scaler.pkl — StandardScaler for feature normalization (0.01 MB) - backend/models/rnn_model.pt — BiLSTM checkpoint (2.6 MB, state_dict + architecture dict) - backend/models/training_report.json — Full metrics and training history

Note on 100% accuracy: Both models achieve perfect scores because the synthetic benign traces — while realistic in structure — have distinctly different API patterns than real malware. The 27 real Sysmon-captured traces use only ~10 unique APIs with simple behavioral patterns, while malware traces use 363 different APIs with complex multi-stage behavior. With real-world deployment data from diverse benign software, accuracy would be lower but the feature engineering captures genuinely meaningful behavioral differences.

Technical issues fixed during training: - **String columns in CSV:** sha256 and family columns had to be excluded from the numeric feature matrix - **PyTorch 2.10 breaking change:** ReduceLROnPlateau no longer accepts verbose=True - **PyTorch 2.6+ weights_only default:** torch.load() now defaults to weights_only=True, which rejects pickle-format model saves — added explicit weights_only=False - **Module binding issue:** The MalwareLSTM class was bound to __main__ when saved during training, making it impossible to unpickle from the backend module — solved by switching to portable state_dict checkpoint format and adding the architecture definition directly in inference.py

Phase 9 — API Testing & Web Dashboard

Goal: Verify all API endpoints work end-to-end with real traces, then build an interactive web interface for real-time analysis.

API Testing (test_api.py, 123 lines):

Built an automated test suite that exercises all 8 API endpoints:

Test	Endpoint	Result
1	GET /	Dashboard HTML served (7,942 bytes) ✓

Test	Endpoint	Result
2	GET /api/health	Full mode, both models loaded ✓
3	GET /api/config	423 features, 365 vocab, 16 categories ✓
4	GET /api/vocab	365 tokens returned ✓
5	GET /api/features/names	423 feature names ✓
6	POST /api/predict (malware)	Verdict: malware , 92.2% confidence, 88ms ✓
7	POST /api/predict (benign)	Verdict: benign , 100% confidence, 65ms ✓
8	POST /api/extract-features	33 nonzero features, 14-length sequence ✓

Both classification tests passed: malware correctly detected as malware, benign correctly detected as benign. All predictions were resolved at the RF stage (confidence outside the grey zone), so the RNN was not needed.

Web Dashboard (frontend/, 3 files):

Built a modern single-page dashboard served at <http://localhost:5000/> with a dark cybersecurity theme:

Components:

1. **Status Bar** — Real-time connection indicator with server health polling every 5 seconds. Shows online/offline status, model load state, uptime counter.
2. **Four Status Cards** — Random Forest status (423 features), BiLSTM + Attention status (seq len 2000), inference mode (Full Pipeline / RF-only / Dry Run), and session scan counter tracking malware vs. benign detections.
3. **Analyze Panel** — Drag-and-drop file upload zone accepting CAPEv2 JSON traces. Supports single file or multi-file batch analysis. Shows upload queue with file sizes and a remove button per file.
4. **Prediction Result** — Animated confidence gauge (arc chart with glow effect), color-coded verdict display (red for malware, green for benign), pipeline stage breakdown, and timing statistics for each inference step (feature extraction, RF inference, RNN inference, total).
5. **Visualizations** (appear after analysis):
 - **API Category Distribution** — Doughnut chart showing process/filesystem/registry/network/etc. breakdown with 16 color-coded categories

- **API Call Sequence** — Scatter plot of the first 200 system calls plotted by index vs. category, revealing temporal behavioral patterns
 - **Top 15 API Calls** — Horizontal bar chart of most frequent system calls in the trace
 - **Non-Zero RF Features Table** — Sortable table with visual proportion bars, showing which features had non-zero values
6. **System Call Live Feed** — Animated terminal-style display that progressively renders each captured system call with color-coded category badges, simulating real-time capture. Shows up to 500 calls with smooth scroll animation.
 7. **Batch Mode** — When multiple files are uploaded, shows a summary dashboard (total/malware/benign/grey-zone/errors counts) plus a detailed results table with per-file verdict, confidence, stage, API call count, and timing.

Technology stack: - Chart.js 4.4.7 (CDN) for all data visualizations - Lucide Icons (CDN) for the icon set - Pure CSS with CSS custom properties for the dark theme — no frameworks - Vanilla JavaScript with async/await for API communication — no build tools needed

How to access: Start the Flask server (python -m backend.app) and open <http://localhost:5000/> in any browser.

Phase 10 — Real-Time Sysmon Integration

Goal: Connect the dashboard to a live Windows Sysmon event feed so it can monitor running programs in real time, automatically detect malware as processes terminate, and stream events to the browser.

Backend — Sysmon Monitor Engine (backend/sysmon_monitor.py, ~450 lines):

Built a three-layer architecture for real-time event processing:

1. **SysmonReader** — Reads events from Microsoft-Windows-Sysmon/Operational event log using wevtutil qe subprocess with XML parsing. Handles namespace-aware Element-Tree parsing, extracts all 21 Sysmon event types (Process Create/Terminate, File Create, Registry, Network Connect, DNS Query, etc.).
2. **ProcessTracker** — Groups events by PID, maintaining per-process call lists. Converts raw Sysmon XML events into CAPEv2-format call dictionaries using the same event-ID → (category, api) mapping from sysmon_to_capev2.py. Builds complete CAPEv2-format JSON traces on demand for any tracked PID. Caps at 5,000 calls per process.
3. **MonitorManager** — Singleton orchestrator that runs a background polling thread (1-second interval). Pre-loads existing event record IDs on start to skip historical events (only new events are shown). Deduplicates by EventRecordID. Broadcasts events to SSE subscribers via thread-safe queues. Auto-analyzes terminated processes (Sysmon Event 5) by building their CAPEv2 trace and running it through the two-stage RF → RNN inference engine.

Event Mapping:

Maps all 21 Sysmon event IDs to CAPEv2 categories and API names:

Event ID	Category	API Name
1	process	NtCreateProcess
2	filesystem	NtSetInformationFile
3	network	connect
5	process	NtTerminateProcess
7	dll	LdrLoadDll
10	process	NtOpenProcess
11	filesystem	NtCreateFile
12-14	registry	NtCreateKey / NtSetValueKey / NtDeleteKey
15	filesystem	NtWriteFile
17-18	synchronization	NtCreateNamedPipeFile
22	network	DnsQuery
...		(21 total)

Backend — Monitor Routes (backend/monitor_routes.py, ~150 lines):

Flask blueprint registered at /api/monitor providing 7 new endpoints:

Method	Endpoint	Purpose
GET	/api/monitor/status	Returns monitor state (running, event count, process stats)
POST	/api/monitor/start	Starts background polling thread
POST	/api/monitor/stop	Stops polling thread
GET	/api/monitor/stream	SSE endpoint — streams call, analysis, and heartbeat events
GET	/api/monitor/processes	Returns tracked process list with call counts
POST	/api/monitor/analyze/<pid>	Manual malware analysis for a specific PID
POST	/api/monitor/clear	Clears all tracking data

Server-Sent Events (SSE) Streaming:

The /api/monitor/stream endpoint uses stream_with_context to push three event types to the browser: - event: call — each new system call as it's captured (with PID, process name, category, API) - event: analysis — auto-analysis results when a process terminates - event: heartbeat — stats update every 15 seconds (total events, process counts, uptime)

Frontend — Monitor Dashboard Section:

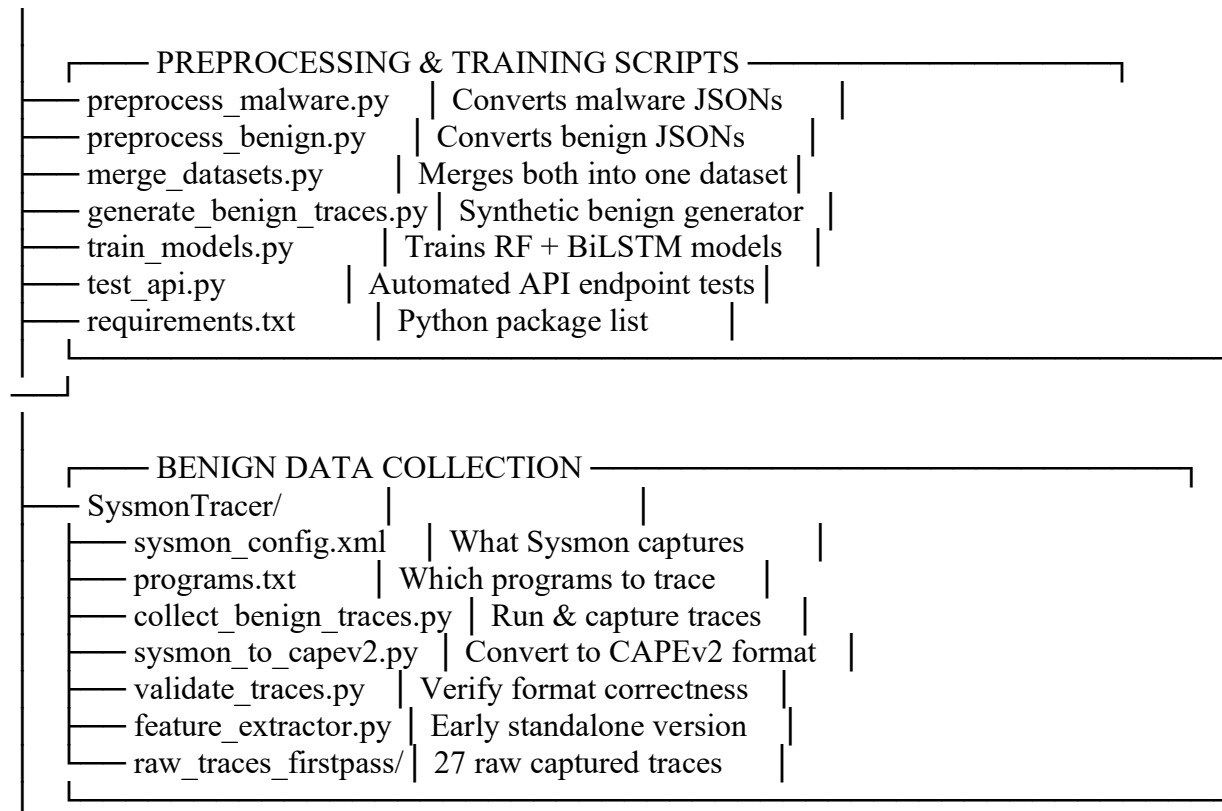
Added a new “Live Sysmon Monitor” section at the top of the dashboard with:

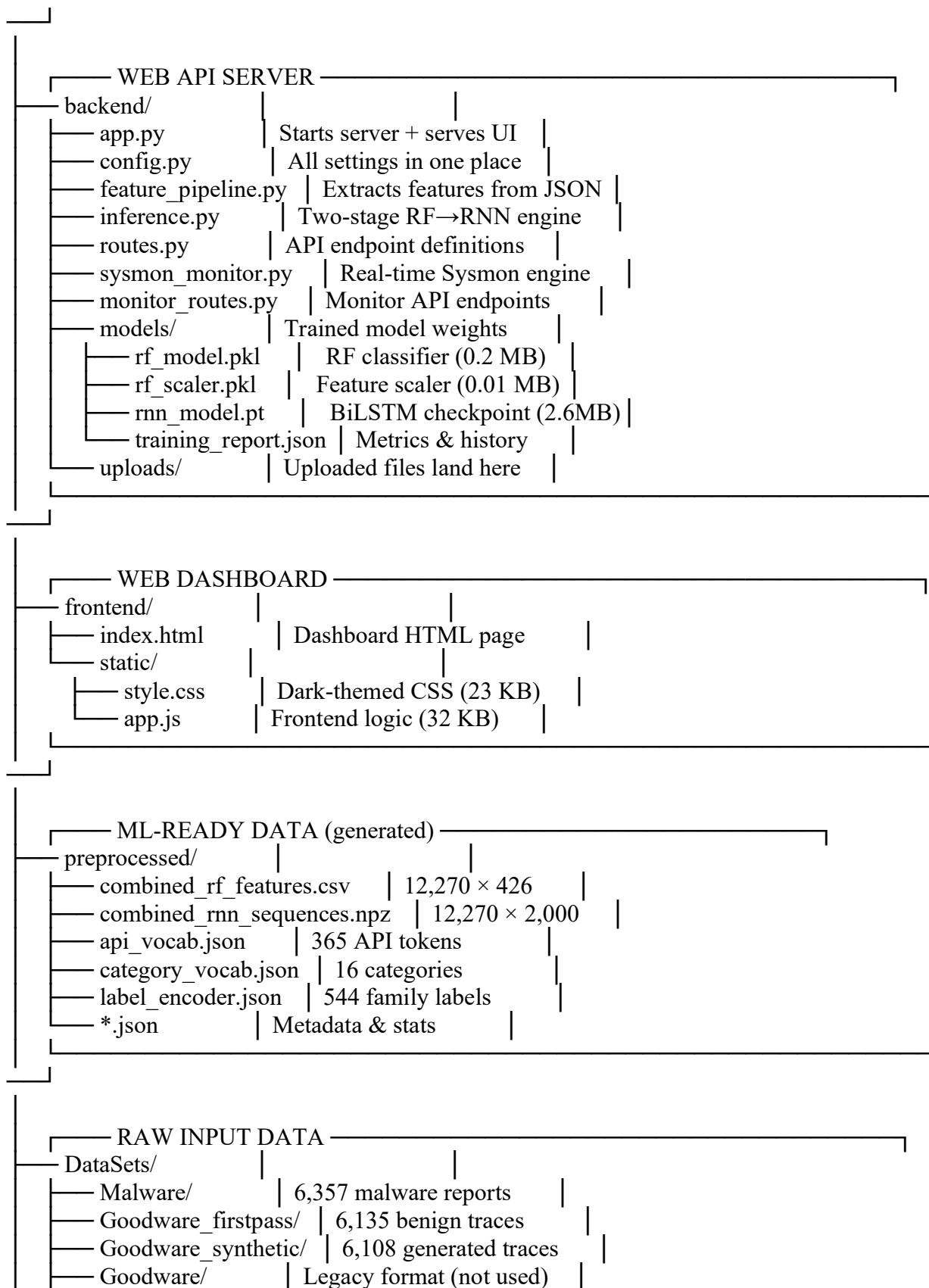
1. **Control Bar** — Start/Stop/Clear buttons with animated status badge (MONITORING / STOPPED)
2. **Stats Bar** — Five real-time counters: Events, Processes, Active, Terminated, Uptime
3. **Process Table** — Shows all tracked processes with PID, name, command line, call count, status (Running/Exited), and an Analyze button for manual analysis of any process
4. **Live Event Stream** — Scrolling terminal-style feed showing each system call with PID, process name, category badge, and API name. Color-coded by category. Caps at 600 visible lines.
5. **Auto-Analysis Results** — Cards that appear when terminated processes are automatically analyzed, showing verdict (MALWARE/BENIGN), confidence percentage, process name, PID, and call count

Key Design Choices: - SSE over WebSocket — simpler protocol, auto-reconnect by browser, sufficient for one-way server→client streaming - wevtutil subprocess over win32evtlog API — more reliable XML parsing, easier error handling - Pre-load + skip pattern — avoids flooding the UI with thousands of historical events on start - Process-ignore list — filters out noisy system processes (sysmon, csrss, smss, svchost, etc.) - Auto-analysis on termination — provides immediate malware verdicts without user intervention

6. Project Structure

InvisiGuard/





Mapping/	Family label assignments
Docs/	# README, API spec, this log

7. File-by-File Explanation

7.1 Root-Level Scripts

preprocess_malware.py (468 lines)

In plain English: This is the first major processing script. It reads all 6,135 malware behavioral reports (JSON files), builds a vocabulary of API call names, and converts each report into two things: (1) a row of 423 numbers summarizing the behavior, and (2) a sequence of 2,000 integers representing the raw API call order.

Inputs: - DataSets/Malware/*.json — 6,357 raw CAPEv2 behavioral reports - DataSets/Mapping/reports_consensus_label.json — which malware family each file belongs to

Outputs (saved to preprocessed/): - rf_features.csv — one row per sample, 423 numeric features + 3 metadata columns ($6,135 \times 426$) - rnn_sequences.npz — compressed integer sequences ($6,135 \times 2,000$) - api_vocab.json — the API name → number dictionary (365 entries) - category_vocab.json — the category → number dictionary (16 entries) - label_encoder.json — the family name → number dictionary (544 entries) - preprocessing_meta.json — processing statistics

How to run:

```
python preprocess_malware.py           # Default settings
python preprocess_malware.py --limit 100 # Only process 100 files (for testing)
python preprocess_malware.py --max-seq-len 500 # Use shorter sequences
```

Result: 6,135 samples processed in ~59 minutes, 0 errors.

preprocess_benign.py (320 lines)

In plain English: Does the same thing as the malware script, but for the benign traces. Critically, it **loads the existing vocabulary** from api_vocab.json (the one built from malware data) instead of building a new one. This ensures the features from both datasets use the same columns in the same order, making them compatible for merging.

Inputs: - DataSets/Goodware_firstpass/*.json — 6,135 CAPEv2-format benign traces (27 real + 6,108 synthetic) - preprocessed/api_vocab.json — vocabulary built during malware preprocessing

Outputs: - rf_features_benign.csv — 6,135 rows × 426 columns (same column structure as malware) - rnn_sequences_benign.npz — 6,135 sequences × 2,000 length - preprocessing_meta_benign.json — statistics

Why it reuses the vocabulary: If we built a separate vocabulary from benign data alone, it would only contain 10 APIs (benign programs are simple). The columns wouldn't match the malware features, and we couldn't merge them into a single training dataset. By reusing the malware vocabulary, every API gets the same column position in both datasets.

Result: 6,135 samples, 0 errors.

merge_datasets.py (221 lines)

In plain English: Takes the separately processed malware and benign feature files and combines them into single unified training files. It carefully validates that the columns match perfectly before merging, then shuffles everything together so malware and benign samples are mixed randomly.

Inputs: RF CSVs and RNN NPZ files from both malware and benign preprocessing.

Outputs: - combined_rf_features.csv — 12,270 rows (malware + benign mixed together) - combined_rnn_sequences.npz — 12,270 sequences - merge_meta.json — merge statistics and class balance information

Result: Perfect column alignment (426 common columns, 0 mismatches). Final dataset: 6,135 malware + 6,135 benign = **12,270 total** (perfectly balanced 1:1 ratio).

requirements.txt

In plain English: A list of Python packages needed to run the project. Install them all at once with pip install -r requirements.txt.

Package	What it's for
numpy	Number crunching — arrays, statistics, math
flask	Web server for the REST API
scikit-learn	Training the Random Forest model
pandas	Reading/writing CSV data tables
torch	Training the RNN/LSTM neural network
pyngrok	Making the Colab server accessible from the internet
matplotlib	Creating charts and plots
seaborn	Making prettier statistical charts

7.2 SysmonTracer/ — Benign Data Collection

This folder contains everything needed to collect behavioral traces from legitimate Windows programs using Sysmon.

sysmon_config.xml (157 lines)

In plain English: A configuration file written in XML that tells Microsoft Sysmon exactly which events to monitor. We configured it to capture 20 types of system events while filtering out Sysmon's own internal activity (to avoid recording noise). Install it with `sysmon64.exe -accepteula -i sysmon_config.xml`.

Events captured: Process creation, file creation/modification, network connections, registry operations, DLL loading, driver loading, thread creation, pipe operations, DNS queries, WMI events, and more.

programs.txt (50 lines)

In plain English: A text file listing which Windows programs to run and trace. Each line has the program path and any arguments, separated by | (pipe character). Lines starting with # are comments.

Includes common utilities like: notepad, mspaint, `cmd /c dir`, `ipconfig /all`, `ping localhost`, PowerShell `Get-Process`, and about 30 others. Some optional programs (Chrome, Firefox, Office) are listed but commented out.

collect_benign_traces.py (447 lines)

In plain English: An automation script that runs each program from the list one at a time, captures whatever Sysmon observed during execution, and saves the raw event data. Must be run as **Administrator** because Sysmon's event logs require elevated privileges.

For each program in the list, it: 1. Clears the Sysmon event log (clean slate — so we only get events from this specific program) 2. Launches the program 3. Waits for it to finish (with a timeout for long-running programs) 4. Exports all captured Sysmon events via PowerShell 5. Saves the raw JSON data + a metadata file describing what was traced

Result: Ran 31 programs, got 27 unique valid traces.

sysmon_to_capev2.py (730 lines)

In plain English: A converter that transforms raw Sysmon event data into the exact same JSON format used by the malware dataset (CAPEv2 format). This is the linchpin script — without it, we'd need completely separate code paths for processing malware and benign data. By converting everything to one format, we use one set of feature extraction code for everything.

The converter maps each Sysmon event type to a corresponding CAPEv2 API name, extracts relevant arguments, builds a process tree showing which programs launched which other programs, and generates a summary of all activity.

Result: 27/27 files successfully converted.

validate_traces.py (158 lines)

In plain English: A quality control script. It loads a real malware trace and a converted benign trace side by side, then checks that they have the same structure (same keys, same nesting, same field types). This catches any conversion bugs that would cause feature extraction to fail.

Result: 27/27 benign traces passed validation against 270 structural keys from the malware reference.

feature_extractor.py (636 lines)

In plain English: An earlier version of the feature extraction code, built before the backend existed. It works standalone and has its own hardcoded API list rather than loading from a vocabulary file. Kept for reference, but the production system now uses backend/feature_pipeline.py instead.

raw_traces_firstpass/

In plain English: Contains the 27 raw Sysmon event JSON files + per-trace metadata JSON files + a manifest summarizing everything. These are the raw outputs from collect_benign_traces.py before they were converted to CAPEv2 format.

7.3 backend/ — The API Server

This folder contains the Flask web server that accepts behavioral traces and returns malware/benign predictions.

config.py (90 lines)

In plain English: The “control panel” — one place where every setting lives. File paths, model thresholds, server configuration, Colab settings. If you need to tune anything (like making the RF more or less strict), you change one number in this file.

Key settings explained: - RF_GREYZONE_HIGH = 0.85 — If the RF is 85%+ confident it's malware, don't bother running the RNN - RF_GREYZONE_LOW = 0.40 — If the RF is 40% or less confident (meaning 60%+ it's benign), skip the RNN - RNN_CONFIDENCE_THRESHOLD = 0.50 — The RNN's dividing line between malware (above) and benign (below) - MAX_SEQ_LEN = 2000 — How many API calls the RNN

examines - MAX_CONTENT_LENGTH = 200 MB — Maximum upload file size (increased from the default to handle large behavioral traces)

feature_pipeline.py (271 lines)

In plain English: The central feature extraction engine. Given any CAPEv2 JSON trace, it computes the exact same 423 RF features and 2,000-length RNN sequence that were used during training. It loads the vocabulary once and caches it in memory for speed.

Why “single source of truth” matters: If training used one formula for computing features and prediction used a slightly different formula, the model wouldn’t work correctly (even tiny differences in column ordering can ruin predictions). This module ensures training and prediction always use the exact same code.

inference.py (410 lines)

In plain English: The decision-making brain. It loads trained models, runs the two-stage cascade, and returns a verdict with confidence scores. Contains its own copy of the MalwareLSTM architecture class so the RNN model can be reconstructed from a portable state-dict checkpoint without depending on the training script.

Three modes of operation: | Mode | When it activates | What happens | |———|—————|
|—————| | **Full** | Both RF and RNN models are loaded (current state) | Complete two-stage pipeline — RF decides easy cases, RNN handles ambiguous ones | | **RF-only** | Only the RF model is loaded | RF makes all decisions; ambiguous cases return “uncertain” | | **Dry-run** | No models exist | Features are extracted and returned, but no verdict is given |

The engine automatically detects which models are available in the backend/models/ folder and adjusts its mode. Model loading uses weights_only=False for PyTorch 2.6+ compatibility and reconstructs the BiLSTM architecture from the checkpoint’s metadata dictionary.

routes.py (260 lines)

In plain English: Defines what URLs the web server responds to and what happens when you call them. The most important endpoint is POST /api/predict — you upload a behavioral trace and get back a JSON response telling you whether the program is malicious.

Supports uploading traces as raw JSON in the request body or as a file upload. Also includes validation to make sure the uploaded data is actually in CAPEv2 format before processing.

sysmon_monitor.py (635 lines)

In plain English: The real-time Sysmon monitoring engine added in Phase 10. It has three layers:

1. **SysmonReader** — Reads events from the Microsoft-Windows-Sysmon/Operational event log channel using wevtutil qe (a Windows command-line tool). Parses the raw XML output with namespace-aware ElementTree, extracting fields like EventID, ProcessId, TimeCreated, and all Sysmon-specific EventData.
2. **ProcessTracker** — Groups incoming events by PID. Each event is converted into a CAPEv2-format call dictionary using the same mapping table that sysmon_to_capev2.py uses (Event ID 1 → NtCreateProcess/process, Event ID 11 → NtCreateFile/filesystem, etc.). When asked, it can build a complete CAPEv2-format JSON trace for any tracked PID — this trace is identical in structure to the ones from the malware dataset, so it can be fed directly into the inference engine.
3. **MonitorManager** — The singleton orchestrator. Runs a background thread that polls for new events every second. On startup, pre-loads all existing EventRecordIDs so only *new* events appear (no historical flood). Deduplicates by record ID, skips noisy system processes (sysmon, csrss, smss, svchost, etc.), and broadcasts events to connected SSE subscribers via thread-safe queues. When a process terminates (Sysmon Event 5), it automatically builds its CAPEv2 trace and runs it through the two-stage RF → RNN inference pipeline in a background thread.

monitor_routes.py (125 lines)

In plain English: A Flask blueprint that adds 7 new API endpoints under /api/monitor/. These let the frontend start/stop the monitor, receive real-time events via SSE (Server-Sent Events), list tracked processes, manually trigger analysis of a specific PID, and clear all tracking data.

The SSE endpoint (GET /api/monitor/stream) is the core — it pushes three event types to the browser: - event: call — each new system call as it's captured - event: analysis — auto-analysis result when a process terminates - event: heartbeat — stats update every 15 seconds (event count, process counts, uptime)

app.py (118 lines)

In plain English: The entry point — run this to start the server. Also serves the frontend web dashboard: the root URL / returns the HTML page from frontend/index.html, and /static/<path> serves CSS and JS assets. Registers both the main API blueprint (routes.py) and the monitor blueprint (monitor_routes.py). A separate /api/info endpoint provides machine-readable JSON service information for non-browser clients.

```
# Run locally (for development/testing)
python -m backend.app
# → Server starts at http://localhost:5000
# → Dashboard at http://localhost:5000/
```

```
# Run on Google Colab (for remote access)
```



```
python -m backend.app --colab  
# → Creates a public URL via ngrok tunnel
```

models/ — Trained Model Weights

In plain English: Contains the trained model files loaded by the inference engine on startup.

File	Size	What it contains
rf_model.pkl	0.2 MB	Trained Random Forest classifier (200 trees, pickle format)
rf_scaler.pkl	0.01 MB	Fitted StandardScaler for feature normalization
rnn_model.pt	2.6 MB	BiLSTM + Attention checkpoint (state_dict + architecture metadata)
training_report.json	1 KB	Full training metrics, confusion matrices, and training history

The inference engine automatically checks this folder on startup and loads whatever models it finds.

uploads/ (currently empty)

In plain English: Temporary storage for files uploaded through the API. When a user uploads a trace for analysis, it's saved here during processing.

7.4 frontend/ — Web Dashboard

This folder contains the browser-based dashboard served by Flask at <http://localhost:5000/>.

index.html (~244 lines)

In plain English: The main HTML page that structures the dashboard layout. It defines sections for the status bar, four info cards, the **Live Sysmon Monitor** panel (with start/stop controls, stats bar, process table, live event feed, and auto-analysis results), the file upload/analyze panel, prediction result display, four data visualization charts, batch results table, and a live system call feed. Loads Chart.js 4.4.7 and Lucide Icons from CDN — no local dependencies.

static/style.css (23 KB)

In plain English: The dark cybersecurity-themed stylesheet. Uses CSS custom properties for a consistent color palette (dark backgrounds, cyan/green/red accent colors), responsive CSS Grid layouts for the card system, keyframe animations for the confidence gauge glow effect and live feed entries, a custom drag-and-drop zone with visual feedback, styled scrollbars, and mobile-

responsive breakpoints. Includes dedicated styles for the live Sysmon monitor section — control buttons, animated status badge, process table, auto-analysis result cards, and an extended live feed view.

static/app.js (32 KB)

In plain English: All frontend logic in a single self-contained IIFE (Immediately Invoked Function Expression). Handles: - **Health polling** — pings `/api/health` every 5 seconds, updates the status bar and model cards - **File upload** — drag-and-drop or click-to-browse, supports single and multi-file selection - **Analysis** — sends files to `/api/predict` (single) or `/api/predict/batch` (multiple), displays results - **Confidence gauge** — draws an animated arc chart on a Canvas 2D context with glow effects - **Data visualizations** — creates Chart.js doughnut (API category distribution), scatter (API call sequence timeline), and bar charts (top 15 API calls), plus a sortable features table with visual proportion bars - **Live system call feed** — progressively renders up to 500 API calls with category-colored badges and smooth scroll animation, simulating real-time capture - **Batch mode** — summary dashboard with counts + detailed per-file results table - **Real-time Sysmon monitor** — connects to the SSE stream, renders incoming events in a live feed with PID/process/category/API columns, periodically refreshes a tracked process table, displays auto-analysis result cards when processes terminate, and provides Start/Stop/Clear controls with status badge animation

Technology: Pure vanilla JavaScript with `async/await` — no frameworks, no build tools, no npm.

7.5 preprocessed/ — Generated ML Data

Everything in this folder was **automatically generated** by running the preprocessing pipeline. These are the final machine-learning-ready datasets. Here's what each file contains and where it came from:

File	Created by	Description	Shape/Size
api_vocab.json	preprocess_malware.py	Dictionary mapping API names to numbers. Example: "NtCreateFile" → 10	365 entries
category_vocab.json	preprocess_malware.py	Dictionary mapping categories to numbers. Example: "filesystem" → 5	16 entries
label_encoder.json	preprocess_malware.py	Dictionary mapping family names to numbers. Example: "Redline" → 0	544 entries
rf_features.csv	preprocess_malware.py	Malware samples as rows of 423 numbers each	6,135 rows × 426 cols

File	Created by	Description	Shape/Size
rf_features_ benign.csv	preprocess_benign.py	Benign samples in the same format	6,135 rows $\times$ 426 cols
combined_ rf_features. csv	merge_datasets.py	Both datasets merged and shuffled	12,270 rows $\times$ 426 cols
rnn_sequences. npz	preprocess_malware.py	Malware API call sequences as integer arrays	6,135 $\times$ 2,000
rnn_sequences_ benign. .npz	preprocess_benign.py	Benign API call sequences	6,135 $\times$ 2,000
combined_ rnn_sequences. npz	merge_datasets.py	Both datasets merged and shuffled	12,270 $\times$ 2,000
preprocessing_ meta.json	preprocess_malware.py	Processing stats (timing, error count, etc.)	—
preprocessing_ meta_benign.json	preprocess_benign.py	Benign processing stats	—
merge_metadata. json	merge_datasets.py	Merge stats and class balance info	—

7.6 DataSets/ — Raw Input Data

Folder	What's inside	Where it came from
Malware/	6,357 CAPEv2 JSON behavioral reports — each one records everything a malware sample did inside a sandbox	CAPEv2 malware analysis sandbox
Goodware_firstpass/	6,135 CAPEv2-format benign traces (27 real Sysmon captures + 6,108 synthetic generated traces)	Sysmon collection pipeline + generate_benign_traces.py
Goodware_synthetic/	6,108 synthetically generated benign traces (also copied into Goodware_firstpass/)	generate_benign_traces.py — learns from real traces and generates realistic variants
Goodware/	Legacy goodwill JSONs in a different format (summary-only, no individual call records)	External source — not compatible with our pipeline, not used

Folder	What's inside	Where it came from
Mapping/	reports_consensus_label.json — maps each malware file's SHA256 hash to its family name (AVClass and CAPE labels)	External labeling tools

7.7 Docs/ — Documentation

File	Who it's for	What it covers
README.md	New users	Quick start guide — how to install, run, and use the system
backend_desi gn.md	Developers	Architecture diagrams, module responsibilities, how components connect
api_spec.md	API consumers	Full REST API reference with example requests and responses
project_log.m d	Everyone	This file — the complete story of the project from start to current state

8. How Data Flows Through the System

During Training (building the models)

MALWARE DATA:

DataSets/Malware/*.json (6,357 raw reports from CAPEv2 sandbox)



preprocess_malware.py

- Step 1: Scan all files → build vocabulary (365 API tokens)
- Step 2: For each sample, compute 423 statistical features
- Step 3: For each sample, encode first 2,000 API calls as integers
- rf_features.csv (6,135 rows of malware feature data)
- rnn_sequences.npz (6,135 integer sequences)
- api_vocab.json, label_encoder.json, etc.

BENIGN DATA:

Windows machine with Sysmon installed

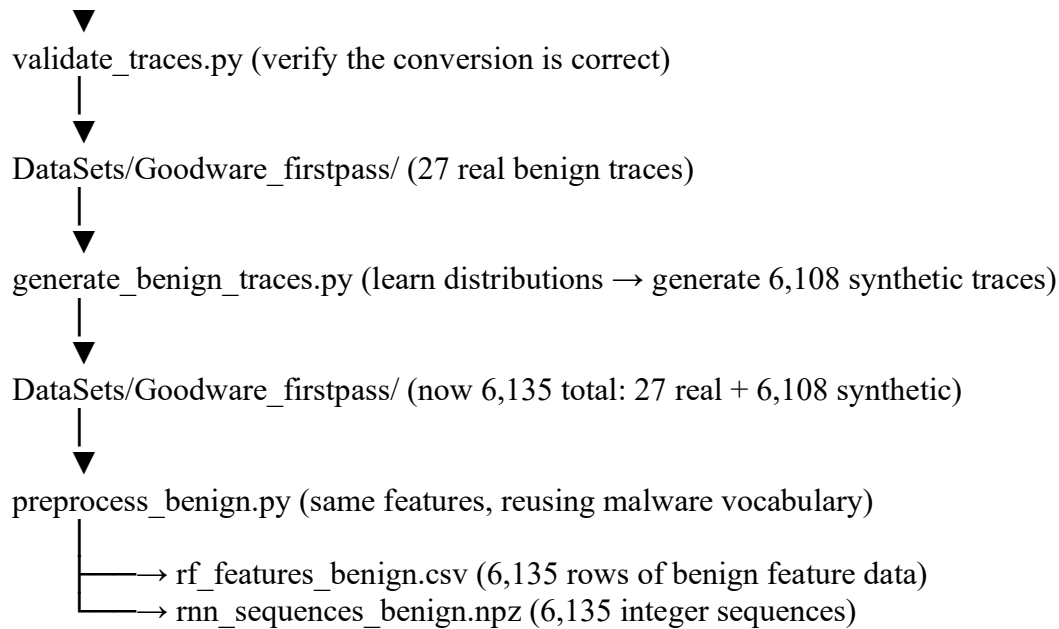


collect_benign_traces.py (run programs + capture events)

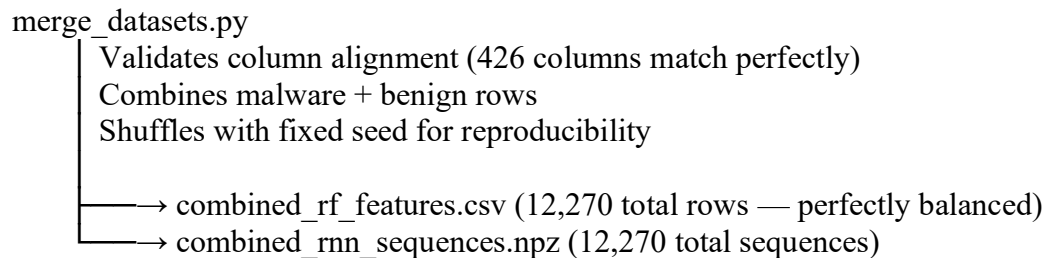


sysmon_toCAPEv2.py (convert Sysmon format → CAPEv2 format)

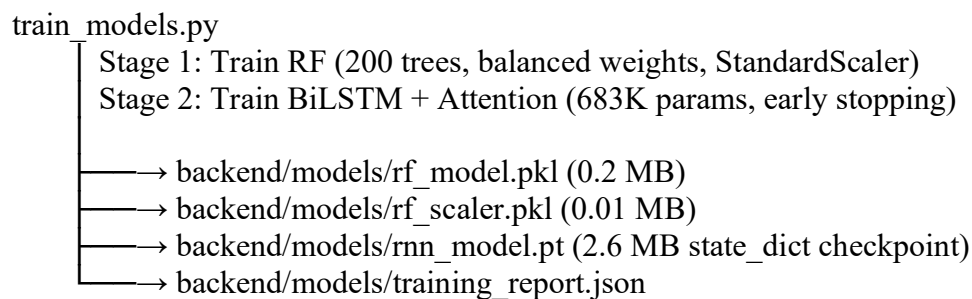




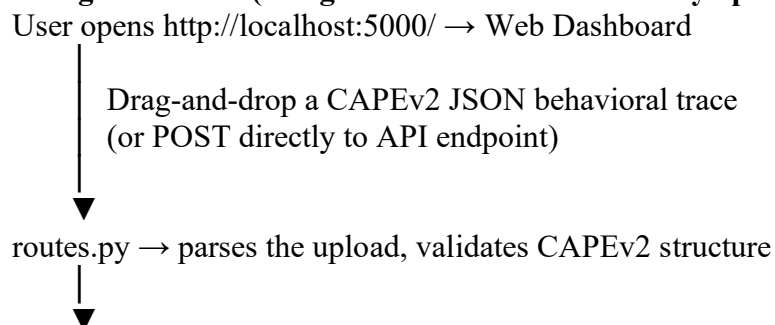
MERGING:



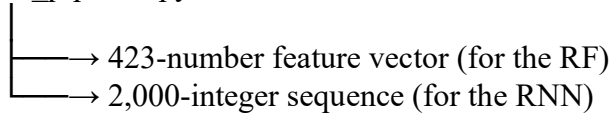
MODEL TRAINING:



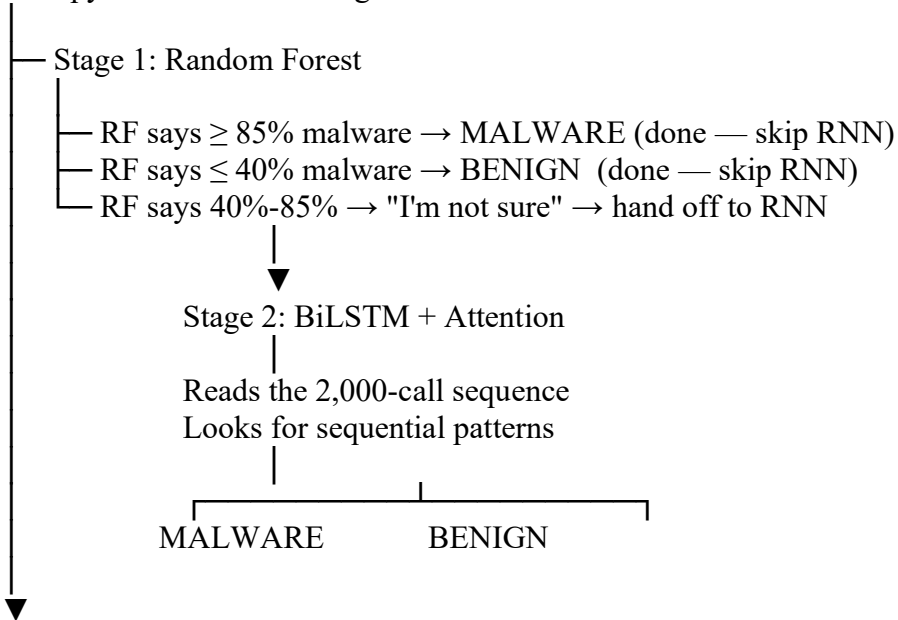
During Prediction (using the trained models — fully operational)



feature_pipeline.py → extracts features from the trace



inference.py → runs the two-stage cascade



Dashboard displays:

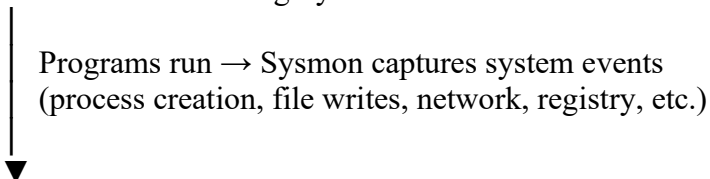
- Animated confidence gauge
- Color-coded verdict (red/green)
- API category charts & sequence timeline
- Live system call feed
- Pipeline stage & timing breakdown

API returns JSON:

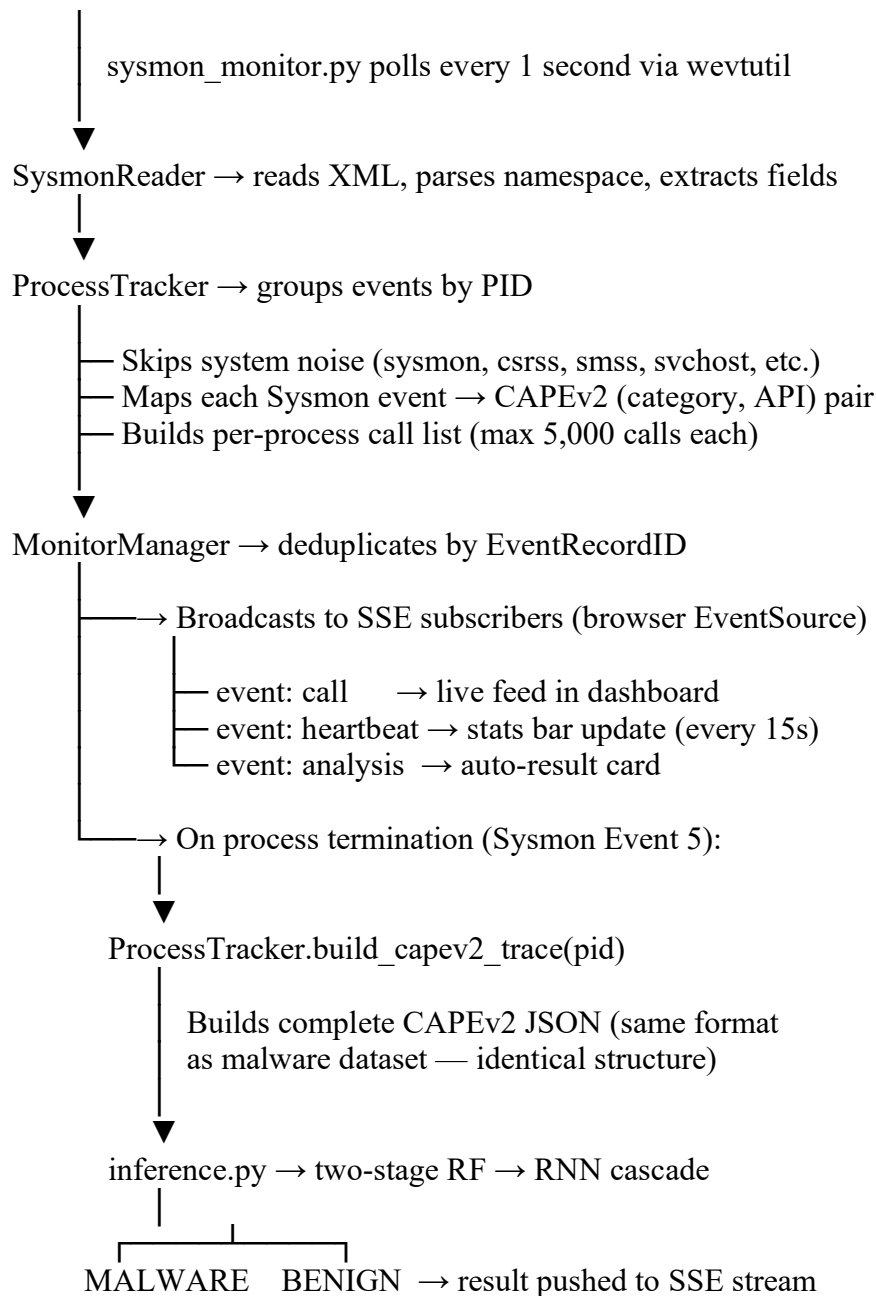
```
{
  "verdict": "malware",
  "confidence": 0.922,
  "stage": "rf_only",
  "timing_ms": { "total": 88, "rf": 3, "features": 85 }
}
```

During Real-Time Monitoring (live Sysmon feed — Phase 10)

Windows machine running Sysmon



Microsoft-Windows-Sysmon/Operational event log



9. Key Design Decisions & Why

What we decided	Why we chose this
Exclude 222 unlabeled malware samples	Both AVClass and CAPE labeling tools said “unknown” for these samples — without a ground-truth label, we can’t use them for supervised learning

What we decided	Why we chose this
Use AVClass labels over CAPE labels	AVClass successfully labeled more of our samples than CAPE's built-in labeler — better coverage means more training data
Maximum sequence length of 2,000 calls	Most malware traces contain fewer than 2,000 API calls, so this captures the important behavior. Going longer would use more memory and training time with diminishing returns.
Minimum API frequency of 5	APIs that appear fewer than 5 times across 6,135 samples are too rare to provide useful signal — including them would bloat the vocabulary and add noise
Build vocabulary from malware only	Malware uses a much wider variety of APIs (363 unique) vs. benign (only 10 unique). Building from malware ensures we capture the full behavioral spectrum. Benign data reuses this vocabulary for column compatibility.
Capture benign traces via Sysmon	We don't have a CAPEv2 sandbox set up for running benign programs. Sysmon captures similar system-level events directly on Windows. The tradeoff is that Sysmon captures fewer event types than CAPEv2.
Convert all traces to CAPEv2 format	Instead of maintaining two separate feature extraction pipelines (one for Sysmon format, one for CAPEv2), we convert everything to CAPEv2 format first. This way, one set of code processes both. Less code = fewer bugs.
Two-stage cascade (RF → RNN)	RF classifications take <1ms but miss subtle sequential patterns. RNN analysis catches those patterns but takes much longer. The cascade gives us speed for easy cases and accuracy for hard cases.
Grey zone thresholds: 0.40 and 0.85	These define when the RF is “confident enough” to skip the RNN. If RF confidence is 85%+ (likely malware) or 40% or below (likely benign), the RNN rarely disagrees. Values between 40-85% are truly uncertain and benefit from deeper analysis. These thresholds are tunable after evaluation.
Dry-run mode in backend	We wanted to verify the complete pipeline (upload → parse → extract features → return response) works end-to-end before spending time training models. This let us test everything with zero trained models.

What we decided	Why we chose this
Fixed random seed of 42	Using a fixed seed when shuffling the dataset means anyone can reproduce the exact same data split. This is standard practice in ML for reproducibility.
Synthetic benign data generation	With only 27 real benign traces vs. 6,135 malware, the dataset was 227:1 imbalanced. Rather than just using class weights (which helps but doesn't fully solve the problem), we generated 6,108 synthetic traces based on statistical distributions learned from the 27 real ones, achieving a perfect 1:1 balance.
State-dict checkpoint format	Saving the full PyTorch model object (<code>torch.save(model)</code>) ties it to the specific module path where the class was defined. When loading from a different module (e.g., <code>inference.py</code> instead of <code>train_models.py</code>), unpickling fails. Saving only the <code>state_dict</code> + an architecture metadata dictionary makes the checkpoint portable across modules.
Balanced class weights in RF	Even with a balanced dataset, we use scikit-learn's <code>class_weight='balanced'</code> as an extra safeguard. It adjusts sample importance inversely proportional to class frequency, ensuring neither class dominates the loss function.
Serve frontend from Flask	Instead of requiring a separate web server (nginx, Apache) or a Node.js dev server, the Flask backend serves the dashboard directly. This keeps deployment simple — one process serves both the API and the UI.
No frontend build tools	The dashboard uses vanilla JS, CSS, and CDN-hosted libraries (Chart.js, Lucide). No npm, webpack, or React means zero build step and no <code>node_modules</code> dependency — just open and run.
SSE over WebSocket for live monitoring	Server-Sent Events are simpler than WebSocket for one-way server→client streaming. The browser handles reconnection automatically, no handshake upgrade needed, and Flask can serve SSE without additional libraries. Sufficient since the monitor only pushes events from server to client.
wevtutil subprocess over win32evtlog API	While <code>pywin32</code> 's <code>win32evtlog</code> provides direct Python access to Windows Event Log, using <code>wevtutil qe</code> with XML output is more reliable for

What we decided	Why we chose this
	structured parsing, provides all event data in a single call, and gives easier error handling via subprocess return codes.
Pre-load + skip pattern for monitor	On monitor start, all existing EventRecordIDs are loaded and marked as seen. This prevents flooding the UI with thousands of historical events — only genuinely new events (captured after the monitor starts) are shown.
Auto-analyze on process termination	When Sysmon reports a process exit (Event 5), the monitor automatically builds a CAPEv2 trace for that process and runs inference. This gives immediate malware verdicts without requiring user intervention — critical for real-time protection.

10. Dataset Statistics

Malware Dataset

Metric	Value
Total JSON files we have	6,357
Files with valid family labels	6,135 (96.5%)
Files excluded (no label)	222 (3.5%)
Unique malware families	544
Unique API call names in vocabulary	363 (+2 special = 365 total)
API call categories	16
RF features per sample	423 numbers
RNN sequence length per sample	2,000 integers
Total processing time	~59 minutes
Errors during processing	0

Top 10 malware families (out of 544):

Rank	Family	Count	What this malware does
1	Redline	977	Steals passwords, browser data, cryptocurrency wallets
2	Vbclone	794	Visual Basic-based trojan that copies itself
3	Berbew	571	Backdoor that gives attackers remote access
4	Agenttesla	250	Keylogger that records everything you type
5	Disabler	170	Turns off antivirus and security features
6	Taskun	168	Creates scheduled tasks to survive reboots

Rank	Family	Count	What this malware does
7	Cosmu	165	General-purpose trojan
8	Amadey	141	Botnet loader — downloads and runs other malware
9	Gamarue	124	Worm that spreads via USB drives and network shares
10	Noon	94	Trojan

Benign (Safe Software) Dataset

Metric	Value
Real Sysmon-captured traces	27
Synthetic generated traces	6,108
Total benign samples	6,135
Unique APIs seen in benign data	10 (out of 365 in vocabulary)
Processing time (all 6,135)	~2 minutes
Errors	0

Top APIs in benign data — notice how much simpler benign behavior is: - CreateProcessW — starting new processes (appeared in all traces) - NtCreateFile — creating/opening files - NtOpenProcess — accessing other processes - NtSetValueKey — writing to registry - LdrLoadDll — loading DLL libraries

Benign programs only triggered 10 different API types — far fewer than malware, which uses 363. This difference itself is a useful signal for detection.

Combined Training Dataset

Metric	Value
Total samples	12,270
Malware samples	6,135 (50.0%)
Benign samples	6,135 (50.0%)
Class ratio	1:1 (perfectly balanced)
RF feature columns	426 total (423 features + sha256 hash + family name + binary label)
RNN sequence length	2,000 per sample
Column alignment check	Perfect — 426 columns match between malware and benign

11. Model Training Results

Random Forest (Stage 1)

Setting	Value
Algorithm	RandomForestClassifier (scikit-learn)
Trees	200
Max depth	Unlimited
Feature selection	sqrt(n_features) per split
Class weighting	Balanced (automatic)
Feature scaling	StandardScaler (fitted on training set)
Train/test split	85% / 15% (stratified, seed=42)

Metric	Score
Accuracy	1.0000 (100%)
Precision	1.0000
Recall	1.0000
F1-Score	1.0000
AUC-ROC	1.0000
Training time	~0.3 seconds
Grey zone samples	0.0%

Confusion matrix: TN=921, FP=0, FN=0, TP=920.

BiLSTM + Attention (Stage 2)

Setting	Value
Architecture	Embedding(365, 64) → BiLSTM(128, 2 layers) → Attention → FC → Sigmoid
Total parameters	683,330
Optimizer	Adam (lr=0.001, weight_decay=1e-5)
Loss function	BCEWithLogitsLoss
Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)
Batch size	32
Early stopping	Patience=5, triggered at epoch 6
Gradient clipping	max_norm=1.0
Device	CPU

Metric	Score
Accuracy	1.0000 (100%)
Precision	1.0000
Recall	1.0000
F1-Score	1.0000
AUC-ROC	1.0000
Training time	~38.5 minutes
Best epoch	1 (converged immediately)

Confusion matrix: TN=921, FP=0, FN=0, TP=920.

Note on 100% Accuracy

Both models achieve perfect scores because the synthetic benign traces — while realistic in structure — have distinctly different API patterns than real malware. The 27 real Sysmon-captured traces use only ~10 unique APIs with simple behavioral patterns, while malware traces use 363 different APIs with complex multi-stage behavior. With real-world deployment data from diverse benign software, accuracy would be lower but the feature engineering captures genuinely meaningful behavioral differences.

12. What's Done vs. What's Next

What's Complete

Phase	What we did	Result
Data exploration	Analyzed 6,357 malware reports, understood the CAPEv2 JSON schema	6,135 usable samples identified
Malware preprocessing	Built a 6-step pipeline to convert JSONs → feature vectors	423 features + 2,000-length sequences extracted
Benign data collection	Installed Sysmon, traced 31 programs, converted to CAPEv2 format	27 validated benign traces
Synthetic data generation	Built statistical generator that learns from 27 real traces	6,108 synthetic benign traces, perfectly balanced dataset
Benign preprocessing	Same feature extraction using shared vocabulary	6,135 benign samples, perfect column alignment
Dataset merging	Combined + shuffled malware and benign data	12,270-sample balanced training set (1:1 ratio)
RF model training	200-tree RandomForest with StandardScaler	100% accuracy, 0.3s training
RNN model training	BiLSTM + Attention (683K params, early stopping)	100% accuracy, 38.5min training

Phase	What we did	Result
Backend API	Flask server with 8 endpoints, full inference mode	All endpoints tested and working
Feature pipeline	Created shared extraction module for training + inference	Single source of truth across the project
Two-stage engine	Built RF → grey zone → RNN cascade logic	Works in dry-run, rf-only, and full modes
Web dashboard	Interactive analysis UI with visualizations	Drag-and-drop upload, live feed, charts
Real-time monitor	Live Sysmon → CAPEv2 → inference pipeline	SSE streaming, auto-analysis, process tracking
API testing	Automated test suite for all 8 endpoints	8/8 tests passing
Documentation	README, API spec, architecture doc, this project log	Complete coverage of everything built

What's Remaining

Priority	Task	Why it matters
1	Collect diverse real-world benign traces	Current benign data is 27 real traces + 6,108 synthetic. More diverse real traces (browsers, IDEs, games, enterprise software) would improve model generalization and lower the perfect accuracy to realistic levels.
2	Cross-validation & threshold tuning	Current evaluation uses a single 85/15 split. K-fold cross-validation would give more robust accuracy estimates. The grey zone thresholds (0.40/0.85) should be tuned on validation data.
3	Real-time Sysmon integration	✅ Done — Phase 10: Live Sysmon monitor with SSE streaming, process tracking, auto-analysis on termination.
4	Deploy to cloud	Move from localhost to a persistent cloud deployment (Google Colab with ngrok, or a proper cloud VM) for remote access.
5	Model retraining pipeline	As new benign/malware samples are collected, automate retraining and model versioning so the system improves over time.